Enterprise Evaluation Framework for Agentic Skills

Nov 10, 2026 · @Faisal Fareed

Recommendation in brief

Adopt a federated evaluation model: the platform team owns the evaluation machinery, and each SME team owns the definition of "good" for its skill.

Six decisions make this work:

1. **Every skill ships with an eval contract.** A golden dataset, scorers, thresholds, a risk tier, and a named owner. No contract, no registration on the platform.
2. **Three tiers of evaluation, run in order.** Deterministic checks first, LLM judges second, human review third. Each tier only sees what the previous tier passed.
3. **CI gates block regressions.** Every change to a skill, an agent, or the platform runs the relevant evals before promotion.
4. **Production feeds the test set.** Live traces are scored, low scorers go to SME review, and confirmed failures become new golden cases.
5. **Three tools, one job each.** LangSmith or Braintrust is the system of record for datasets, judges, human review, and scorecards. Harbor runs end-to-end agent tasks in sandboxes. AgentCore Policy enforces runtime rules on tool calls.
6. **Policy prevents, evaluation measures.** Rules that must never be broken are enforced at the gateway, outside agent code. Evals then confirm the rule held and measure everything policy cannot express.

The worked example throughout is a Public Filings Analysis skill for equity researchers. Agents retrieve company filings, combine them with internal and public data, and draft a research report.

Who owns what

The platform team owns everything that must be identical across skills. SME teams own everything that requires domain judgment.

Asset	Owner	Notes
Tracing standard and trace tags	Platform	Every trace carries skill ID, version, and team
CI pipeline and release gates	Platform	Reads each skill's contract and runs its evals
Shared scorer library	Platform	Numeric reconciliation, citation integrity, faithfulness judge
Guardrail scorers (PII, policy, entitlements)	Platform with Compliance	Mandatory for every skill, no opt-out
Agent evals (routing, tool use, cost budgets)	Platform	Includes the cross-skill routing set
Harbor harness, base environments, data snapshots	Platform	Sandboxes, frozen filing snapshots, tool stubs
Harbor task instructions and verifiers	SME team with Platform	SMEs define the task and pass criteria, Platform packages them
Runtime policies on tool calls	Compliance and Security	Authored in natural language or Cedar, enforced by Platform at the gateway
Risk tiers and minimum thresholds	Platform with Compliance	Tier sets gate strictness and human sampling rate
Skill and its eval contract	SME team	One named eval owner per team
Golden dataset	SME team	Includes a hard-case split
Judge rubrics and skill thresholds	SME team	Thresholds may exceed the platform minimum, never fall below
Human review queue and judge calibration	SME team	Senior practitioners label, not engineers
Scorecard across all skills	Platform	Comparable numbers for leadership

Two rules keep the boundary clean. SME teams never write pipeline code. The platform team never decides whether a research report is good.

Evaluation design

Each skill is evaluated in three tiers against an SME-owned golden dataset. The example below is the Public Filings Analysis skill.

Golden dataset

Start with 50 to 100 cases. Each case holds a ticker, a filing period, a research question, reference figures, and an SME checklist of material points.

Keep three splits:

- **Smoke** (about 15 cases): runs on every commit, finishes in minutes.
- **Full**: runs before any release.
- **Hard cases**: restatements, amended filings, fiscal year mismatches, non-GAAP adjustments, segment reorganisations.

Tier 1: Deterministic checks

Hard gate. A release needs a 100% pass rate on these.

Check	What it verifies
Numeric accuracy	Every figure reconciles to the XBRL value for the right period, unit, and scale
Recomputation	Margins, growth rates, and ratios recalculate correctly from sourced inputs
Citation integrity	Each claim cites a filing and section, and the cited passage exists
Recency	The latest filing was used, including amendments
Compliance	Restricted list respected, required disclosures present, internal data used only where entitled
Trajectory	Correct tools called, within step and cost budgets

Tier 2: LLM as judge

Threshold gate. Scores must clear the skill threshold and must not regress against the current baseline.

Judge	Question it answers
Faithfulness	Is each claim supported by its cited passage, judged claim by claim?
Completeness	Are the material items on the SME checklist covered?
Change detection	Were period-over-period shifts in MD&A and risk factors caught?
Reasoning and balance	Does the thesis follow from the evidence, with fact separated from opinion?

Use binary pass or fail per criterion with a written rationale. Binary judges are far easier to calibrate against analysts than 1 to 5 scales.

Tier 3: Manual checks

SMEs do four things, and only these four:

- **Calibrate judges.** Senior analysts label 30 to 50 outputs. A judge is trusted once it agrees with them 85 to 90% of the time.
- **Review releases.** Pairwise comparison of the new version against the baseline on borderline cases.
- **Sample production.** All low judge scores plus a random sample go to the review queue.
- **Capture edits.** The diff between the agent draft and the published report is the richest signal available. Feed it back into the dataset.

Run order

Tier 1 runs on every case because it is cheap. Tier 2 runs only on cases that pass Tier 1. Tier 3 sees only borderline and sampled cases. This controls judge cost and stops a fluent report with a wrong number from scoring well.

First-time setup

Build the framework around one real skill, then extract the standard from what worked. The filings skill is a good pilot because its numbers are verifiable.

1. **Instrument tracing.** Send every agent run to the chosen tool as OpenTelemetry traces, tagged with skill ID, version, team, and environment. Nothing else works without this.
2. **Read real outputs before writing any scorer.** The platform lead and two senior analysts review 30 to 50 traces together and list the failure modes they see. This list drives the scorers.
3. **Build the golden dataset.** Analysts write the first 50 cases, with reference figures and material-point checklists. Add the hard-case split.
4. **Write Tier 1 scorers.** Start with numeric reconciliation and citation integrity. They catch the most damaging errors at the lowest cost.
5. **Write and calibrate Tier 2 judges.** One judge per failure mode from step 2. Analysts label a calibration set, and the judge prompt is tuned until agreement reaches the bar.
6. **Set the baseline.** Run the full dataset once and record the scores. Thresholds come from this run, not from aspiration.
7. **Wire the CI gate.** Smoke split on every commit, full split before release. Regression against baseline blocks promotion.
8. **Package end-to-end tasks in Harbor.** Convert 10 to 20 golden cases into Harbor tasks with a frozen filing snapshot and the Tier 1 scorers as the hidden verifier. Run each several times to measure variance.
9. **Put runtime policy in front of the tools.** Compliance writes the tool-access rules. Run them in log-only mode against test traffic, fix false denials, then switch to enforce.
10. **Turn on production scoring.** Reference-free judges and guardrail scorers on sampled live traces, with low scorers routed to the analyst review queue.
11. **Close the loop.** Confirmed production failures and analyst edits are added to the golden dataset on a fixed schedule.
12. **Extract the standard.** Turn what the pilot built into the eval contract template, the shared scorer library, the Harbor task template, and the onboarding checklist.

A reasonable target is steps 1 to 7 in six to eight weeks with one platform engineer and part-time access to two analysts. Steps 8 and 9 add two to three weeks. Steps 10 to 12 follow once the skill is in production.

Tool mapping

LangSmith or Braintrust is the system of record, and both cover every component in the table. Pick one and standardise on it. Harbor and AgentCore add capabilities neither provides, covered in the next two sections.

Component	LangSmith	Braintrust
Golden dataset	Dataset with splits	Versioned dataset
Tier 1 scorers	Custom code evaluators	Code scorers
Tier 2 judges	LLM-as-judge evaluators	LLM-as-a-judge scorers
Shared library	Workspace-scoped evaluators attached to many projects and datasets	Same scorers reused offline and online
Judge calibration	Align evaluator workflow using labels from an annotation queue	Score the judge against human review labels
Tier 3 review	Annotation queues, single-run and pairwise	Human review with multiple reviewers
Release gate	Experiment run from the SDK in CI, compared to baseline	Experiment run in CI through the GitHub Action or CLI
Production scoring	Online evaluators on runs and threads	Online scoring rules, asynchronous
Team isolation	Workspace per team, project per skill	Project per team or skill

How to choose. LangSmith fits best if the platform is built on LangGraph, since tracing and thread-level evals need little extra work. Braintrust fits best if the platform is framework-agnostic and the team wants a CI-first workflow with OpenTelemetry ingestion.

Before committing. Confirm self-hosting, data residency, and access control options with the vendor. Golden datasets will contain internal estimates and entitled data.

Sources: LangSmith evaluation concepts, LangSmith evaluator alignment, Braintrust evaluation docs, Braintrust glossary.

Where Harbor fits

Harbor is the execution harness for end-to-end agent tasks. It complements LangSmith or Braintrust and does not replace them.

Harbor is an open-source framework from the Terminal-Bench team. It places an agent in a sandboxed container, gives it a task, and grades the result with a check the agent never sees.

What Harbor adds

- **Reproducibility.** Each task carries its own environment, so the agent sees the same filings and tool responses on every run.
- **Hidden verification.** Tests live outside the agent's container. The agent cannot read the pass criteria or edit them.
- **Variance measurement.** A job runs every task several times across agents and models, so pass rate replaces a single lucky run.
- **Agent-agnostic comparison.** The same task set runs against different agents or models, which makes platform upgrades comparable.
- **Scale.** Trials run in parallel locally on Docker or on cloud sandbox providers.

What Harbor does not do

Harbor has no production trace scoring, no SME review queues, and no cross-team scorecard. Those stay in the system of record.

Mapping the filings skill to a Harbor task

Harbor component	Filings skill content
`instruction.md`	The research question, as an analyst would ask it
`environment/`	Container with a frozen snapshot of the filings, internal estimates, and stubbed data tools
`tests/`	The hidden verifier: Tier 1 scorers for numbers, citations, recency, and compliance
`solution/`	An analyst-approved reference report, used to prove the verifier passes a correct answer

Harbor component	Filings skill content
`reward.json`	One score per check, so a failure names the check that broke
Trial	One agent attempt at one research question
Job	All tasks, run several times each, across the agent versions under test

Harbor verifiers can also call LLM judges, so Tier 2 can run inside the task when a self-contained result is wanted.

How it connects

1. Golden cases in the system of record are the source. Harbor tasks are generated from a subset of them.
2. Harbor runs the job in CI before a release, or before a model or platform upgrade.
3. A small script written by the platform team pushes each trial's rewards and trajectory into the system of record as an experiment.
4. Analysts review failures there, in the same queue as everything else.

When to use it

Use Harbor for release gates, model upgrades, and agent comparisons, where reproducibility matters most. Use the lighter dataset experiments in LangSmith or Braintrust for the per-commit smoke run, where speed matters most.

Harbor is at an early version, so pin the version in CI and expect interface changes.

Sources: Harbor on PyPI, Harbor task structure, Harbor tasks overview, Harbor job configuration, DZone walkthrough.

AWS Bedrock AgentCore: evaluation and policy

AgentCore provides two relevant services: Evaluations, a managed scoring service, and Policy, a runtime authorization layer for tool calls. Both are generally available as of March 2026.

AgentCore Evaluations

Evaluations scores agent behavior from OpenTelemetry traces. It works with agents built on frameworks such as Strands and LangGraph, and with agents hosted outside AgentCore Runtime through a CloudWatch log group.

Capability	What AgentCore provides
Built-in evaluators	13 managed LLM judges at session, trace, and tool level, such as Goal Success Rate, Faithfulness, Correctness, and Tool Selection Accuracy
Custom LLM judges	Your own judge model, instructions, and scoring schema
Code evaluators	A Lambda function as the evaluator, for deterministic checks
Ground truth	Expected response, expected tool trajectory, and natural-language assertions
On-demand evaluation	A synchronous API for development and CI gates, with a dataset runner for regression suites
Online evaluation	Continuous sampling of a set share of production traces
Results	CloudWatch dashboards and alarms, with the judge's reasoning beside each score

Three limits to plan around:

- Built-in evaluator prompts and models are fixed. Any calibration to your analysts needs a custom evaluator.
- A single on-demand call or online configuration takes up to 10 evaluators.
- The AWS material reviewed describes no SME annotation queue or pairwise review workflow. Tier 3 stays in LangSmith or Braintrust.

How Evaluations maps to the three tiers

Tier	AgentCore mechanism	Filings skill example
Tier 1 deterministic	Lambda code evaluators	Verify every reported figure matches the XBRL value
Tier 1 trajectory	Built-in trajectory match against expected tool sequence	Confirm the agent pulled the latest filing before drafting
Tier 2 judges	Built-in Faithfulness plus custom judges	Custom completeness judge using the analyst checklist
Tier 3 manual	Not covered	Review queue in the system of record

AgentCore Policy

Policy controls which tool calls an agent may make. It sits at the AgentCore Gateway, outside agent code, and evaluates every agent-to-tool request before allowing or denying it.

- **Deterministic.** Rules are written in Cedar, an open-source authorization language. The same request always gets the same decision.
- **Accessible authoring.** Compliance can describe a rule in plain English. The service converts it to Cedar and validates the result with automated reasoning.
- **Fine-grained.** Conditions can use the caller's identity claims and the tool's input parameters.
- **Default deny.** Once a policy engine is attached to a gateway, anything not explicitly permitted is blocked.
- **Safe rollout.** A log-only mode shows what would be denied on live traffic before enforcement is switched on.
- **Content checks.** Since June 2026, Bedrock Guardrails results can be evaluated inside policy, in selected Regions.
- **Auditable.** Policy decisions are logged through AgentCore Observability.

Policy examples for the filings skill

Rule in plain English	What it prevents
Only users with the research entitlement may call the internal estimates tool	Internal data reaching users without entitlement
Forbid any data pull where the ticker is on the restricted list	Research activity on restricted names
Forbid the publish tool unless the request carries supervisory approval	Unreviewed reports leaving the firm
Permit read tools only for this skill, no write or trade tools	A research agent taking actions outside its remit

How policy and evaluation work together

Policy removes a class of failure at runtime. Evaluation proves the policy works and measures what policy cannot express.

- Each policy gets matching test cases in the golden dataset, including cases that should be denied.
- Policy denials in production are a signal. A rising denial rate for one skill means the agent is attempting things it should not.
- Report quality, completeness, and reasoning remain evaluation concerns. No policy can express them.

Recommendation

If the platform is	Use
On AWS with tools behind AgentCore Gateway	Policy as the enforcement layer for all skills. This is the strongest reason to adopt AgentCore.
On AWS and wanting fewer vendors	Evaluations for online scoring and CI gates, with LangSmith or Braintrust kept for datasets and SME review

If the platform is	Use
Multi-cloud or framework-agnostic	LangSmith or Braintrust for all scoring, since all of them read OpenTelemetry traces. Add Policy only where tools sit behind the Gateway.

Whichever split is chosen, run each release gate in one place only. Two systems scoring the same gate will disagree, and teams will pick the friendlier one.

Sources: AgentCore Evaluations overview, built-in evaluators, Policy general availability, Policy and Cedar, Gateway and log-only mode, Guardrails in policy.

How to scale

Scale comes from the eval contract. Each new team fills in the same template, and the platform CI does the rest.

The eval contract

A small manifest stored with each skill:

- Dataset reference and splits
- Which shared scorers apply, plus any skill-specific ones
- Thresholds per scorer
- Harbor task set reference, for skills at the higher risk tiers
- Tools the skill needs, which Compliance turns into its runtime policy
- Risk tier
- Named eval owner

Onboarding a new team

1. The team nominates an eval owner and completes the contract template.
2. The platform assigns a risk tier with Compliance.
3. Compliance writes the skill's tool policy from the contract and runs it in log-only mode.
4. The team builds its golden dataset and reuses shared scorers wherever they fit.
5. The team calibrates any new judges against its own experts.

6. For higher risk tiers, the platform packages a subset of cases as Harbor tasks using the shared template.
7. The team adds cases to the routing set, including "should not trigger" cases.
8. The skill registers once the full dataset passes the gate and the policy is switched to enforce.

Onboard the second team deliberately as a test of the contract. If credit research cannot meet a template that equity research wrote, fix the template before the third team arrives.

Isolation

Each team gets its own workspace or project. A platform-level space holds shared scorers and cross-skill suites. Datasets with internal estimates stay invisible to teams without entitlement.

Cross-team rules

Shared components are where multi-team setups break. Three rules cover the main cases:

Change	Rule
Platform change (model upgrade, new retrieval agent)	Platform runs every team's golden dataset and Harbor task set before release. Any regression blocks rollout or needs an agreed exception.
New skill added	The skill adds routing cases. The full routing set must still pass, so new skills cannot steal traffic from existing ones.
Shared judge updated	Judges are versioned. Teams pin a version and upgrade on their own schedule after rechecking calibration.
Policy added or changed	The change runs in log-only mode first. It moves to enforce once no team sees unexpected denials.

Risk tiers

Tiers keep cost proportional to risk. Client-facing or regulated skills get strict gates and high human sampling. Internal productivity skills get lighter gates and low sampling.

Promotion of scorers

A scorer one team builds that proves useful moves into the shared library. Credit research should not rebuild the citation checker that equity research already wrote.

How to maintain

An evaluation framework decays unless datasets, judges, and thresholds are refreshed on a schedule. Assign each task a cadence and an owner.

Cadence	Task	Owner
Every commit	Smoke split and Tier 1 checks	Platform CI
Every release	Full dataset, Tier 2 judges, Harbor task set, pairwise review of borderline cases	SME team with Platform
Weekly	Clear the production review queue and triage low scorers	SME team
Weekly	Review policy denials for false blocks and attempted violations	Platform with Compliance
Monthly	Add confirmed failures and analyst edits to the golden dataset	SME team
Monthly	Eval guild: share scorers, failure patterns, and contract changes	Platform, all eval owners
Quarterly	Recalibrate judges against fresh analyst labels	SME team
Quarterly	Refresh Harbor data snapshots and add tasks for new filing periods	Platform with SME team
Quarterly	Review thresholds, risk tiers, and policies, and retire stale cases	Platform with Compliance

Cadence	Task	Owner
On model or platform change	Run every team's golden dataset and Harbor tasks, and recheck judge calibration	Platform

What goes stale, and the fix

- **Datasets.** New filing formats, new accounting standards, and new question types appear. Monthly additions from production keep coverage current.
- **Judges.** A judge model upgrade can silently shift scores. Pin judge model versions and recalibrate on every change.
- **Thresholds.** A scorer that passes 100% for two quarters is no longer telling you anything. Tighten it or replace it with harder cases.
- **Hard cases.** Once the skill handles them reliably, move them to the regression set and write new ones.
- **Harbor environments.** Frozen snapshots age, and stubbed tools drift from the real ones. Rebuild them when a tool's interface changes.
- **Policies.** New tools arrive without rules, and old rules outlive the tools they covered. Default deny makes the first case safe, and the quarterly review catches the second.

Health metrics for the framework itself

Track these on the cross-skill scorecard:

- **Judge agreement with SMEs**, per judge. Below the bar means the judge is not trusted for gating.
- **Escape rate.** Production failures that the offline evals did not catch. This is the most important number.
- **Review backlog.** Queue age per team. A growing backlog means sampling rates exceed SME capacity.
- **Dataset freshness.** Share of cases added in the last two quarters.
- **Run-to-run variance.** Spread of pass rates across repeated Harbor trials. High variance means a single passing run proves little.
- **Policy denial rate**, per skill, split into correct blocks and false blocks.
- **Eval cost and runtime** per skill, so gates stay fast enough that teams do not route around them.

Change control

Treat the eval contract, shared scorers, judge prompts, Harbor tasks, and Cedar policies as versioned code with review. A change to a shared scorer follows the same cross-team rule as a platform change.